



ALMA MATER STUDIORUM UNIVERSITY OF BOLOGNA  
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – DISI

# Large Language Models: The Latest Upgrades

Prof. Gianluca Moro

Tutors: Dr. Giacomo Frisoni, Dr. Lorenzo Molfetta

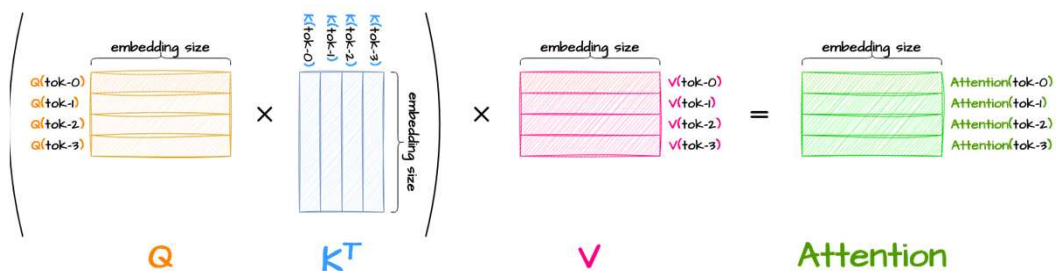
DISI, University of Bologna, Cesena  
Via dell'Università, 50 I-47522 Cesena (FC), Italy  
{name.surname}@unibo.it

## Outline

- **Efficient Attention Mechanisms** 
- Parameter-Efficient Fine-Tuning
- Mixture of Experts
- Optimization & Decoding Strategies
- Architectural Developments
- Multi-modal Language Models

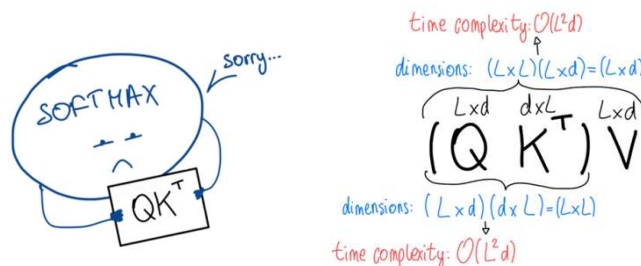
## Attention-related optimization

- Attention mechanism optimizations can be used to further **enhance** the **inference performance** of a Transformer-based model
  - Excluding multi-head attention matrices, the nature of transformer models does not allow any particular parallelizable operation
  - The focus is on efficiency and resource-saving, mainly involving how key, value and query attention matrices are computed

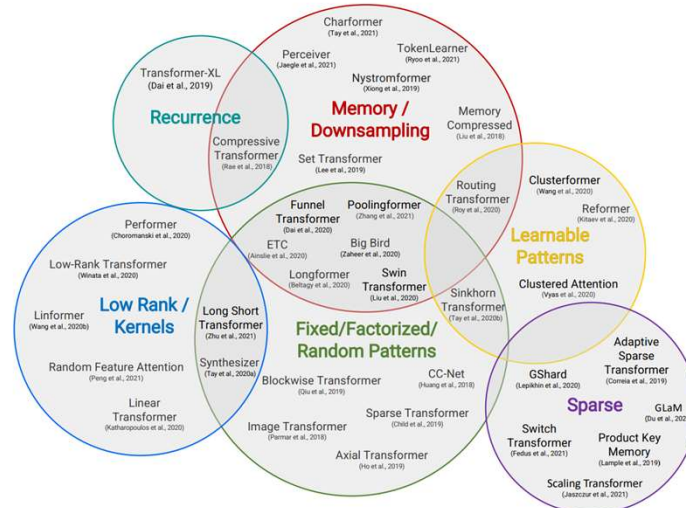


## Efficient self-attention

- Although self-attention is the foundation of the transformer architecture, this operation is somewhat **inefficient**
  - The softmax is the bottleneck of transformers, forcing an  $O(L^2d)$  complexity which is **quadratic** in the sequence length (so dealing with longer sequences become more and more computationally expensive)
- Thus, a plethora of efficient attention variants have been proposed [Tay et al., 2020]; Performer, BigBird, Reformer, Longformer, SMYRF to name a few



## Efficient self-attention: a broad family

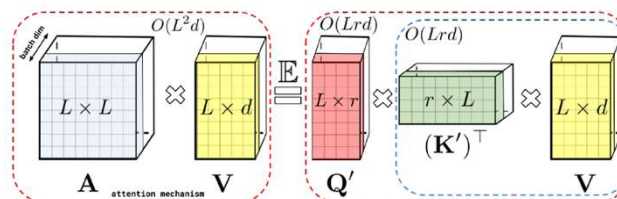


## Performer: a kernel-based example of linear attention

- It uses a function  $\phi$  called *positive random feature map*  $\phi: \mathbb{R}^d \rightarrow \mathbb{R}_+^r$  such that:  

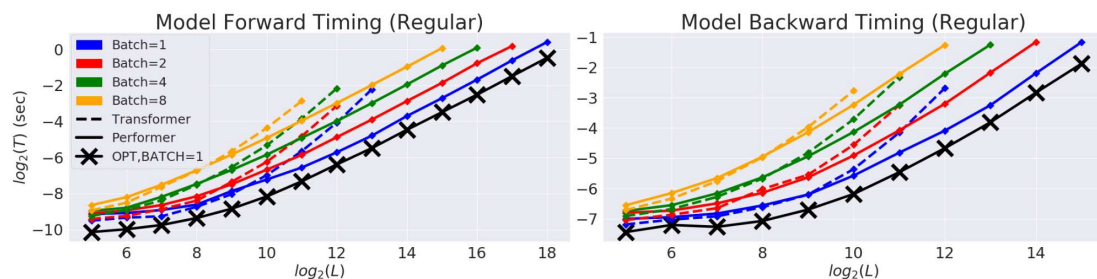
$$K(x, y) = \mathbb{E}[\phi(x)^\top \phi(y)]$$
  - It can be shown that this holds if, given  $r > 0$ , we define  $\phi(x) = \frac{h(x)}{\sqrt{r}} f(Wx)^\top$  where  $W \in \mathbb{R}^{r \times d}$  is a matrix of random orthogonal vectors,  $h(x) = \exp(-\frac{\|x\|^2}{2})$  and  $f = \exp(\cdot)$
- With this trick, Attention is approximated in linear time with respect to the size of the input of length  $L$  by rewriting the formula as follows:  

$$\hat{A}t(Q, K, V) = \hat{D}^{-1} (Q'((K')^\top V)), \quad \hat{D} = \text{diag}(Q'((K')^\top \mathbf{1}_L))$$
  - With  $Q', K' \in \mathbb{R}^{L \times r}$  computed applying the function  $\phi$  to the rows of the matrices  $Q$  and  $K$



## Performer: computational costs

- The log-log graph below shows, as  $L$  varies, the gap between training times for a quadratic attention Transformer (dashed lines) and Performer (solid lines)
- The symbol  $\times$  represents the max achievable speed (using a “dummy” **placebo** attention block that performs nothing)
- Data are shown until the trainings produced a “memory error”



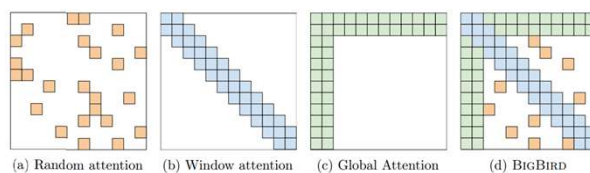
[disai-unibo-nlp.github.io](https://github.com/disai-unibo-nlp)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

7

## BigBird: linear attention based on randomness

- BigBird [[Zaheer et al., 2020](#)] is a model with linear complexity that approximates attention using fixed pattern and memory mechanisms
  - Use sparse graphs in which the nodes are tokens of the input sequence, and the arcs represent the connections in the attention matrix
- BigBird combines 3 types of sparsification:
  - **Random attention:** each node is randomly linked to other  $r$  nodes
  - **Window attention:** each node is connected to its  $w$  neighbors ( $w/2$  on its left and  $w/2$  on its right); this choice comes from the heuristic consideration for which the most important information is generally local information
  - **Global attention:**  $g$  nodes are completely connected; the authors proved that with this type of connections, BigBird becomes a universal approximator of sequence-to-sequence functions and is Turing complete
  - **The input is divided in  $b$  block**, each for example of 64 tokens, and a **quadratic attention is applied only within each block**



[disai-unibo-nlp.github.io](https://github.com/disai-unibo-nlp)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

8

## BigBird: results

- Two models have been proposed
  - BigBird-ITC** (*Internal Transformer Construction*): Global tokens are selected from within the input sequence
  - BigBird-ETC** (*Extended Transformer Construction*): Global tokens are newly added context tokens at the input start
- Improved results in question answering, text classification, summarization
  - Since it can handle longer inputs (i.e., up to 4096 token)
  - BigBird-Pegasus is specifically pre-trained to generate summaries

| Parameter              | BigBird-ITC  | BigBird-ETC  |
|------------------------|--------------|--------------|
| Block length, $b$      | 64           | 84           |
| # of global token, $g$ | $2 \times b$ | 256          |
| Window length, $w$     | $3 \times b$ | $3 \times b$ |
| # of random token, $r$ | $3 \times b$ | 0            |
| Max. sequence length   | 4096         | 4096         |
| # of heads             | 12           | 12           |
| # of hidden layers     | 12           | 12           |
| Hidden layer size      | 768          | 768          |

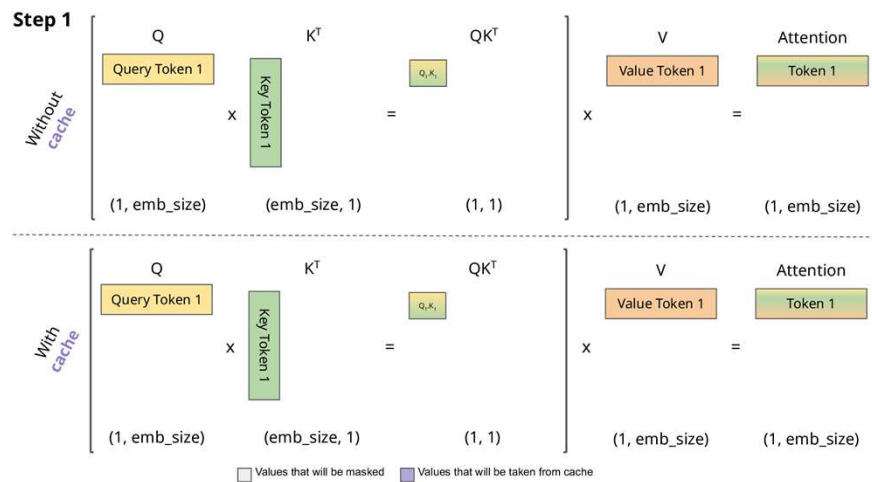
| Model |                    | Arxiv |       |       | PubMed |       |       | BigPatent |       |       |
|-------|--------------------|-------|-------|-------|--------|-------|-------|-----------|-------|-------|
|       |                    | R-1   | R-2   | R-L   | R-1    | R-2   | R-L   | R-1       | R-2   | R-L   |
| Base  | Transformer        | 28.52 | 6.70  | 25.58 | 31.71  | 8.32  | 29.42 | 39.66     | 20.94 | 31.20 |
|       | + RoBERTa          | 31.98 | 8.13  | 29.53 | 35.77  | 13.85 | 33.32 | 41.11     | 22.10 | 32.58 |
|       | + Pegasus          | 34.81 | 10.16 | 30.14 | 39.98  | 15.15 | 35.89 | 43.55     | 20.43 | 31.80 |
|       | BigBird-RoBERTa    | 41.22 | 16.43 | 36.96 | 43.70  | 19.32 | 39.99 | 55.69     | 37.27 | 45.56 |
| Large | Pegasus (Reported) | 44.21 | 16.95 | 38.83 | 45.97  | 20.15 | 41.34 | 52.29     | 33.08 | 41.75 |
|       | Pegasus (Re-eval)  | 43.85 | 16.83 | 39.17 | 44.53  | 19.30 | 40.70 | 52.25     | 33.04 | 41.80 |
|       | BigBird-Pegasus    | 46.63 | 19.02 | 41.77 | 46.32  | 20.65 | 42.33 | 60.64     | 42.46 | 50.01 |

## Key-Value Cache

- Key-Value (KV) caching occurs during multiple token generation steps and only happens in the decoder
- The decoder is causal (i.e., the attention of a token only depends on its preceding tokens)
  - At each generation step, we are recalculating the same previous token attention, when we actually just want to calculate the attention for the new token...
  - ... This is where KV comes into play: **by caching the previous Keys and Values, we can focus on only calculating the attention for the new token**
- With KV caching, the computed matrices are way smaller, which leads to **faster matrix multiplications**
- The downside is that **it needs more GPU VRAM** (or CPU RAM if the GPU is not being used) to cache the Key and Value states

|                                                                                 |               |                     |                |                     |
|---------------------------------------------------------------------------------|---------------|---------------------|----------------|---------------------|
| ⚡ Google Colab, GPT-2, Tesla T4 GPU, time for the generation of 1000 new tokens | w/ KV caching | 11.885 ± 0.272 secs | w/o KV caching | 56.197 ± 1.855 secs |
|---------------------------------------------------------------------------------|---------------|---------------------|----------------|---------------------|

## Key-Value Cache: Visual



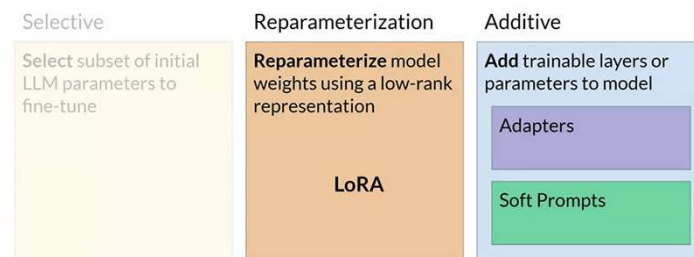
Comparison of scaled dot-product attention with and without KV caching [\[Source\]](#)

## Outline

- Efficient Attention Mechanisms
- **Parameter-Efficient Fine-Tuning** 
- Mixture of Experts
- Optimization & Decoding Strategies
- Architectural Developments
- Multi-modal Language Models

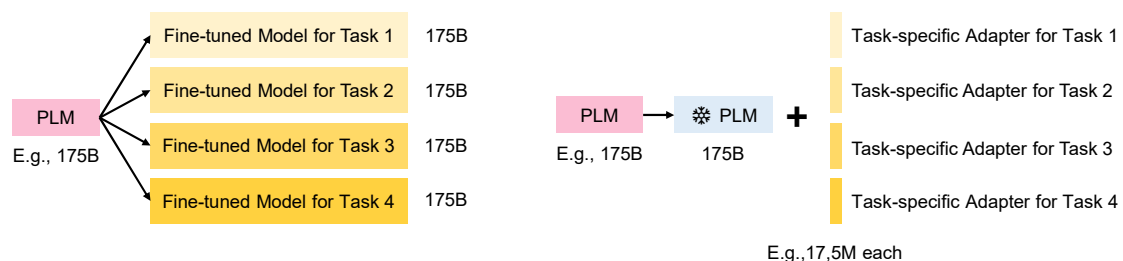
## Parameter-Efficient Fine-Tuning (PEFT)

- PEFT methods enable efficient adaptation of PLMs to various downstream applications without fine-tuning all the model's parameters
  - They **only fine-tune a small number of (extra) model parameters**, greatly decreasing the computational and storage costs
  - State-of-the-art PEFT techniques achieve performance comparable to full fine-tuning, despite reducing the number of trainable parameters (< [0.05%, 5%])
  -  [Extensive community support](#)



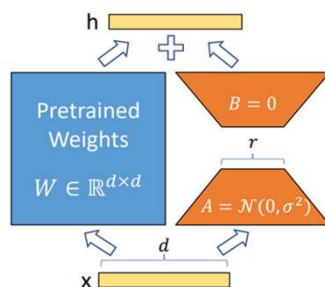
## Adapter Fine-Tuning

- Specialized PLMs are gigantic and need a copy for each downstream task
  - All the downstream tasks can share the same PLM (frozen base), while we add small trainable submodules or reparametrizers (**adapters**) in the Transformer-based architecture
  - ☒ Drastically decreases the task-specific parameters
  - ☒ Less easy to overfit on training data; better out-of-domain performance
  - ☒ Fewer parameters to fine-tune, making them good candidates with small datasets



## Low-Rank Adapters (LoRA): Idea

- Do we need to adjust all LLM parameters during fine-tuning ?
  - LLMs are trained to capture general representations of language and domains
  - When adapting to a specific task, it is reasonable that only few features need to be re-learned
  - Assumption:** LLM weight matrices have a lot of linear dependence, resulting in significantly more parameters than those theoretically required



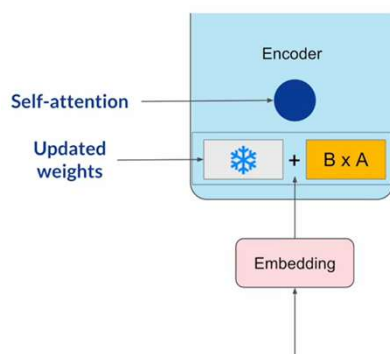
LoRA thinks of fine-tuning not as adjusting parameters, but as **learning factors of the parameter change matrix**

💡 Factor matrices  $A$  and  $B$  are trained to find optimal changes to the pre-trained weights

⚠️ Over-parameterization has been shown to be beneficial in pre-training (which is why modern PLMs are so large). The idea of LoRA is that, once you have learned general knowledge, you can do fine-tuning with much less information.

## LoRA: Target Layers

- Researchers have found that applying LoRA to just the **self-attention layers** is **often enough** to fine-tune for a task and achieve performance gains
  - In principle, you can use LoRA on other components like the FF layers, but...
  - Since most of the parameters are in the attention layers, you get the biggest savings there



- Freeze most of the original LLM weights.
- Inject 2 rank decomposition matrices
- Train the weights of the smaller matrices

Steps to update model for inference:

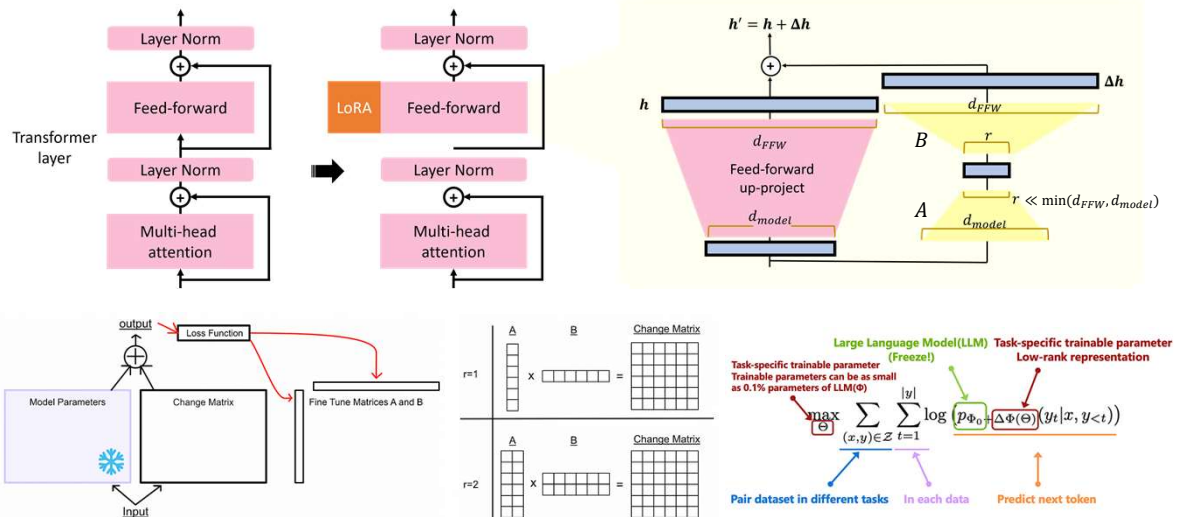
- Matrix multiply the low rank matrices

$$B * A = B \times A$$

- Add to original weights

$$\text{Original Weights} + B \times A$$

## LoRA: Visual Explanation



[disai-unibo-nlp.github.io](https://github.com/disai-unibo-nlp)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

17

## LoRA: Real Numbers

- HuggingFace reports about LoRA effectiveness
  - Single A100 80GB VRAM, more than 64GB CPU RAM

| Model                             | Full Finetuning          | PEFT-LoRA PyTorch       |
|-----------------------------------|--------------------------|-------------------------|
| bigscience/T0_3B (3B params)      | 47.14GB GPU / 2.96GB CPU | 14.4GB GPU / 2.96GB CPU |
| bigscience/mt0-xxl (12B params)   | OOM GPU                  | 56GB GPU / 3GB CPU      |
| bigscience/bloomz-7b1 (7B params) | OOM GPU                  | 32GB GPU / 3.8GB CPU    |

- Using GPT-3 175B as a reference
  - LoRA can reduce the number of trainable params by 10,000 times
  - GPU memory requirement can be reduced by 3 times

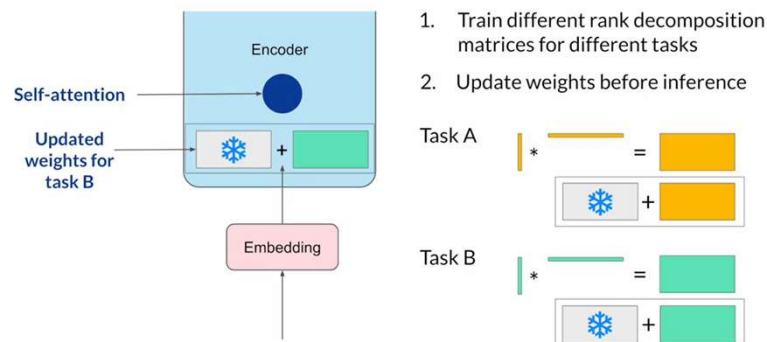
[disai-unibo-nlp.github.io](https://github.com/disai-unibo-nlp)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

18

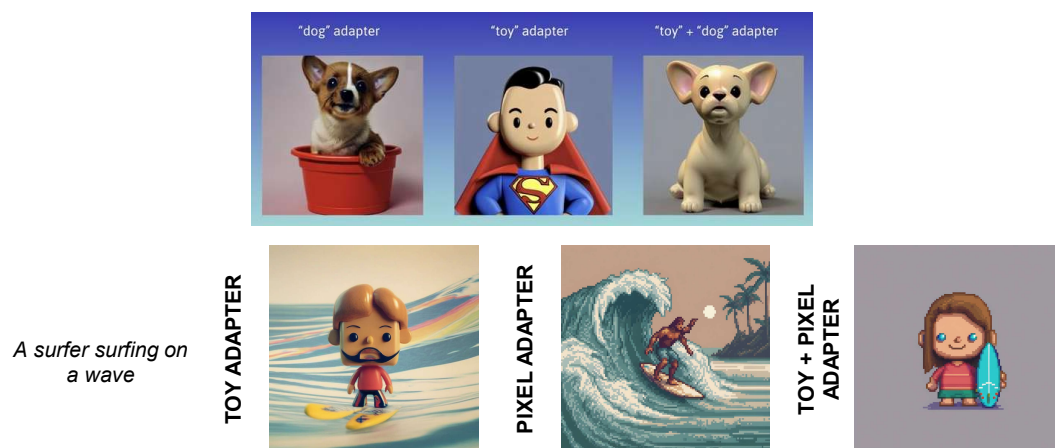
## LoRA: Switching Lego Blocks

- The memory required to store a LoRA adapter is very small
  - You can train adapters for multiple tasks, switch out the weights as needed, and avoid having to store multiple full-size versions of the LLM



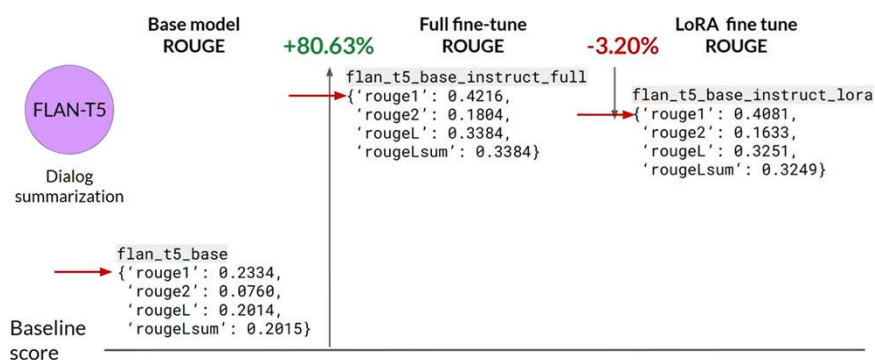
## LoRA: Combining Lego Blocks

- Adapters can also be combined (**multi-adapter training and inference**)
  - Weighted combination [Source] [Diffusion Notebook]



## LoRA: Impact on Effectiveness

- How good are LLMs trained with LoRA ?
  - Example scenario with FLAN-T5 for Dialog Summarization
  - LoRA ROUGE scores are a little lower than full fine-tuning despite training a much smaller number of parameters and using significantly less compute



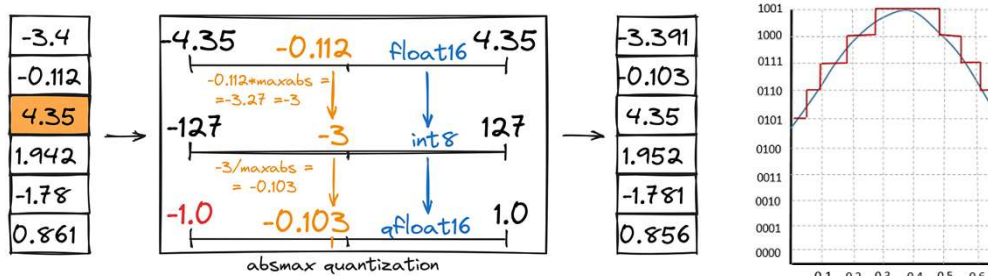
## LoRA: How to Choose the Rank ?

- Still an active research area
- Insights from the original paper [Hu et al., 2021]
  - The authors found a plateau in the loss values for ranks greater than 16
  - The range 4 – 32 can provide you with a good trade-off between reducing the number of trainable parameters and preserving performance

| Rank $r$ | val_loss    | BLEU         | NIST          | METEOR        | ROUGE_L       | CIDEr         |
|----------|-------------|--------------|---------------|---------------|---------------|---------------|
| 1        | 1.23        | 68.72        | 8.7215        | 0.4565        | 0.7052        | 2.4329        |
| 2        | 1.21        | 69.17        | 8.7413        | 0.4590        | 0.7052        | 2.4639        |
| 4        | 1.18        | <b>70.38</b> | <b>8.8439</b> | <b>0.4689</b> | 0.7186        | <b>2.5349</b> |
| 8        | 1.17        | 69.57        | 8.7457        | 0.4636        | <b>0.7196</b> | 2.5196        |
| 16       | <b>1.16</b> | 69.61        | 8.7483        | 0.4629        | 0.7177        | 2.4985        |
| 32       | <b>1.16</b> | 69.33        | 8.7736        | 0.4642        | 0.7105        | 2.5255        |
| 64       | <b>1.16</b> | 69.24        | 8.7174        | 0.4651        | 0.7180        | 2.5070        |
| 128      | <b>1.16</b> | 68.73        | 8.6718        | 0.4628        | 0.7127        | 2.5030        |
| 256      | <b>1.16</b> | 68.92        | 8.6982        | 0.4629        | 0.7128        | 2.5012        |
| 512      | <b>1.16</b> | 68.78        | 8.6857        | 0.4637        | 0.7128        | 2.5025        |
| 1024     | 1.17        | 69.37        | 8.7495        | 0.4659        | 0.7149        | 2.5090        |

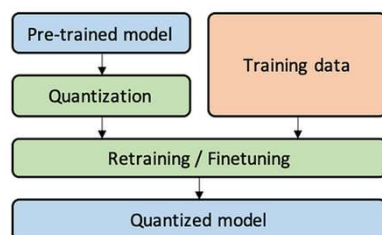
## Quantization

- Quantization is the process of representing a model weight with fewer bits
  - Reduces the memory footprint, the compute resources needed for model serving, and increases speed...
  - ... This often results in information loss (accuracy decrease) that we can recover to an extent by fine-tuning the quantized model a bit more (extra calibration step)



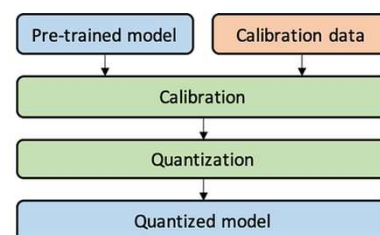
## Quantization Types

- Most algorithms present in the market require a PLM in full precision... What you have to do to create a quantized version ?



### Quantization-Aware Training (QAT)

A pre-trained model is quantized and then fine-tuned using training data to adjust parameters and recover accuracy degradation. The calibration process is often conducted in parallel with the fine-tuning process.

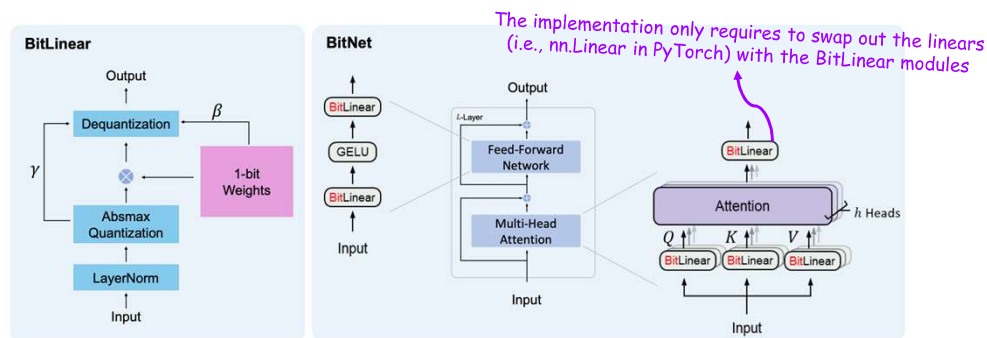


### Post-Training Quantization (PTQ)

First, the model undergoes a calibration step (e.g., on a small subset of training data) to determine the clipping ranges and the scaling factors. Then, it is quantized.

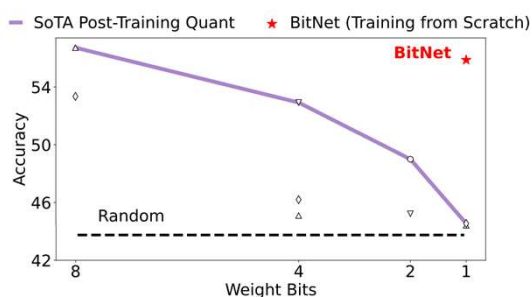
## From-Scratch Quantization: BitNet

- BitNet [Wang et al., 2023] takes a different approach wherein the model is **trained from scratch with quantization**, representing weights with only 1.58 bits
- Maintains performance on par with full-precision Transformer LLMs; remarkable cost-effectiveness in latency, memory usage, and energy consumption



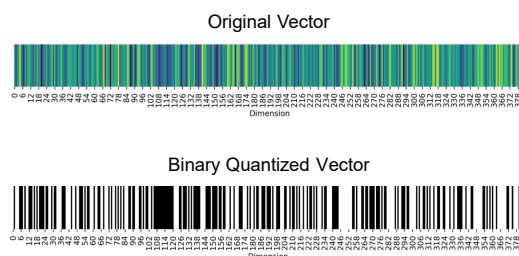
## Binary Quantization

- You can change a vector to 0's and 1's to reduce memory usage by 32x and still have good accuracy; huge potential in VectorDBs [Weaviate Blog]



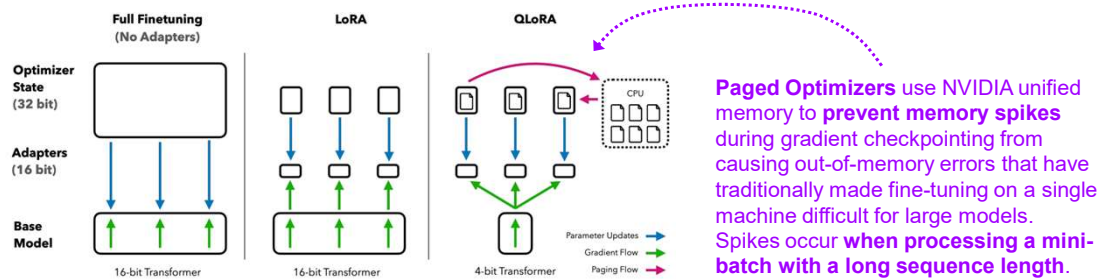
🔗 BitNet b1.58 70B runs 4.1x faster than the LLaMA baseline, saving 71.4x more energy, and handling 11x more data at once

Vector embeddings are secretly hiding barcodes



## QLoRA [Dettmers et al., 2023]

- Simple but brilliant combination of *quantization* and *LoRA*
  - Quantized base model (frozen) + Non-quantized trainable LoRA adapters
  - QLoRA uses **LoRA as an accessory to fix the errors introduced during quantization**
  - During the forward pass and backpropagation, the quantized weights (e.g., 4 bits) are dequantized back to full precision; the actual training occurs in higher precision formats
  - 🚀 The authors train a model that (unquantized) is larger than the GPU: a 65B model (which is 130GB unquantized) on a 48GB card



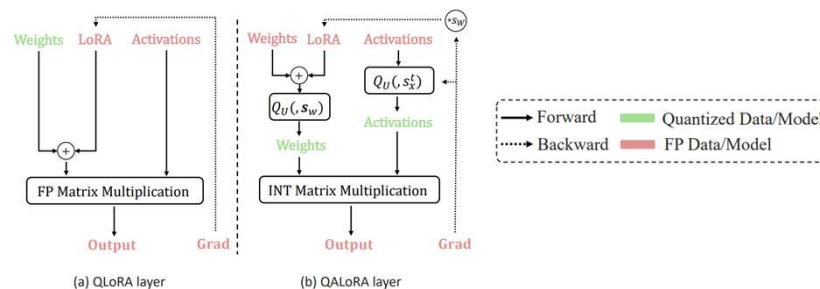
[disi-unibo-nlp.github.io](https://disi-unibo-nlp.github.io)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

27

## QALoRA [He et al., 2023]

- It was mainly released for fine-tuning diffusion models, but can easily be generalized for training any type models, just like LoRA
- QALoRA vs QLoRA
  - The weights of the LoRA adapters are also quantized along with the weights of the model during the fine-tuning process
  - More efficient training as there is no need for the conversion step during backpropagation



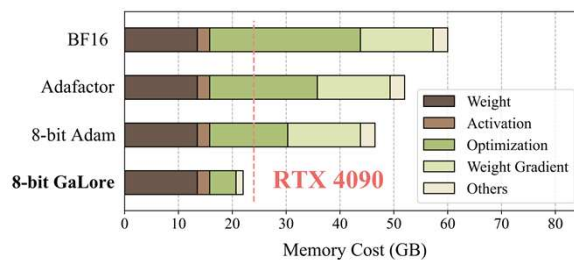
[disi-unibo-nlp.github.io](https://disi-unibo-nlp.github.io)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

28

## GaLore [Zhao et al., 2024]

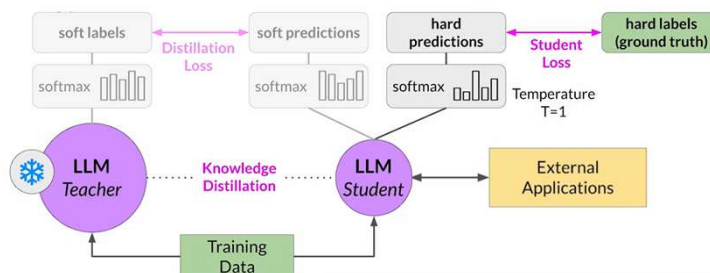
- In contrast to LoRA, GaLore reduces the memory by projecting the **optimizer states and gradients** into a lower-dimensional representation
  - With LoRA, (1) the optimal weight matrices may not be low-rank, (2) the reparameterization changes the gradient training dynamics
  - Up to 82.5% reduction in memory for storing optimizer states
  - It allows to complete a full fine-tuning of LLaMA-2-7B on consumer-size 24GB GPUs



Memory consumption of pre-training a LLaMA-7B model with a token batch size of 256 on a single device, without activation checkpointing and memory offloading

## Knowledge Distillation

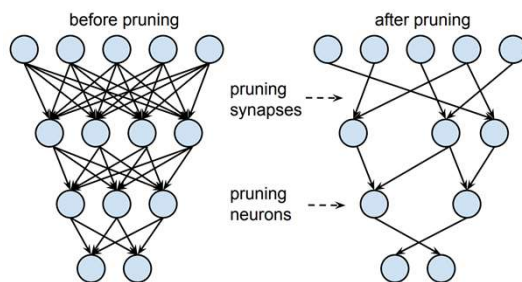
- Train a **small student** model to statistically mimic a **larger teacher** model, either just in the final prediction layer or in the model's hidden layers as well
  - You freeze the teacher model's weights, already fine-tuned on the training data, and use it to generate completions for training data
  - At the same time, you generate training data completions using your student model
  - Two goals: **(1)** Distillation loss to [align soft student-teacher distributions](#) + **(2)** Usual student loss to generate the correct predictions based on your ground-truth training data



⚠ With distillation you are training a second, smaller model to use during inference. You are not reducing the model size of the initial LLM in any way.

## Model Pruning

- Remove less important components without sacrificing performance
  - **Weight pruning:** identify and remove connections (weights) between neurons that contribute less to the overall model performance (e.g., below a certain threshold)
  - **Unit pruning:** instead of pruning individual weights, entire neurons or units are pruned based on their importance (e.g., low activation values or minimal impact on model output)
  - **Structured pruning:** remove entire structures or sub-networks rather than individual weights or neurons (e.g., entire layers based on their importance to model performance)

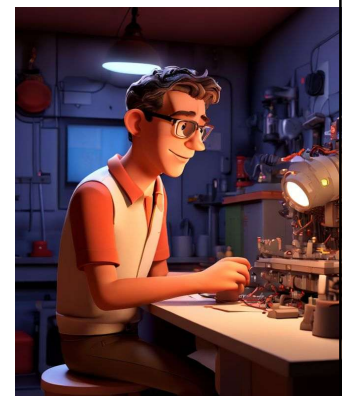


Pruning is often followed by fine-tuning to recover any lost performance and ensure that the model continues to learn and adapt to the data it encounters

Recent contributions are task-agnostic and aim to preserve the multi-task solving and language generation ability of the original LLM [Ma et al., 2023]

## Training on Commodity Hardware

- The limiting factor in training LLMs on consumer-grade GPUs is the amount of memory available on these cards
  - While gaming GPUs like the RTX 3090 or 4090 offer impressive computational power, they typically have max 24GB VRAM...
  - ... This is significantly less than the memory found on data center-class GPUs, such as the A100 or H100, which can have up to 80GB VRAM ← a server rack with 8 of them can easily cost hundreds of thousands of dollars [Source]
  - 💀 A 70B LLM has parameter size of 140GB; just loading it into the GPU can require 2x A100 GPUs with 80GB memory each [Source]
- ❤️ For the first time, by using QLoRA and splitting the model parameters across multiple devices, we can efficiently train a 70B LLM on two 24GB GPUs



[Can you run it?](#)

## Outline

- Efficient Attention Mechanisms
- Parameter-Efficient Fine-Tuning
- **Mixture of Experts** 
- Optimization & Decoding Strategies
- Architectural Developments
- Multi-modal Language Models

## Mixture-of-Experts (MoE): Overview

- Deep learning framework resembling a team of specialists [[Fedus et al., 2022](#)]
  - Imagine a team of experts, each with a specific area of expertise; e.g., a group of doctors, each specializing in different areas of medicine
  - Similarly, MoE employs **a collection of expert models, each trained to excel in a specific sub-region of the overall problem space**
- A variant of the Transformer architecture
  - A MoE block breaks the monolithic pattern and involves multiple sub-networks (experts) for next token prediction
  - A gating function decides to which expert the input representation must be routed to (i.e., which expert is *active*)
  - Experts are combined through a gated linear combination (**collective knowledge**)

Expert networks themselves can be MoEs (**Hierarchical MoE**), leading to deeper and potentially even more efficient architectures [[HuggingFace Blog](#)]



## MoE: Key Components Breakdown

### ▪ Experts

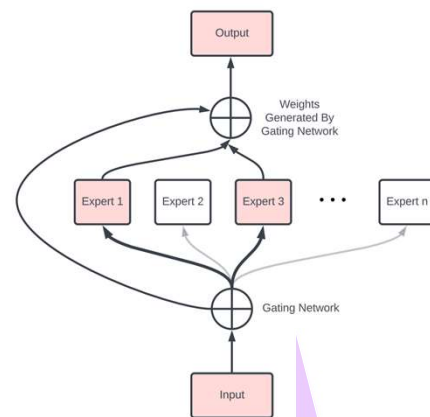
- Individual machine learning models (e.g., feed-forward networks, LLMs, adapters)
- Trained independently on different training data subsets, allowing them to become proficient in handling specific types of input

### ▪ Gating network

- A conductor network dynamically determines which expert(s) are most suitable for a given input, similarly to how a team leader assigns tasks to the most appropriate specialists

### ▪ Combination function

- Once the gating network has selected the relevant experts, their outputs are combined using a function like averaging or weighted averaging



Looks like are talking about physics, which one of you is going to help me predict the next token?

## MoE: Sparse Model

### ▪ The key to MoE's efficiency lies in its **sparsity**

- Each expert can be distributed in parallel over multiple GPUs making inference faster
- Unlike traditional models, MoE activates only a subset of experts for each input
- Since every token has its own expert (or top- $k$  experts) processing it, we technically added more learnable parameters without affecting inference costs
- MoE also mitigates overfitting by preventing over-reliance on specific training data
- By involving only relevant experts, MoEs can potentially learn more robust data representations, leading to improved generalization capabilities

Output of the  $i$ -th expert for the input  $x$

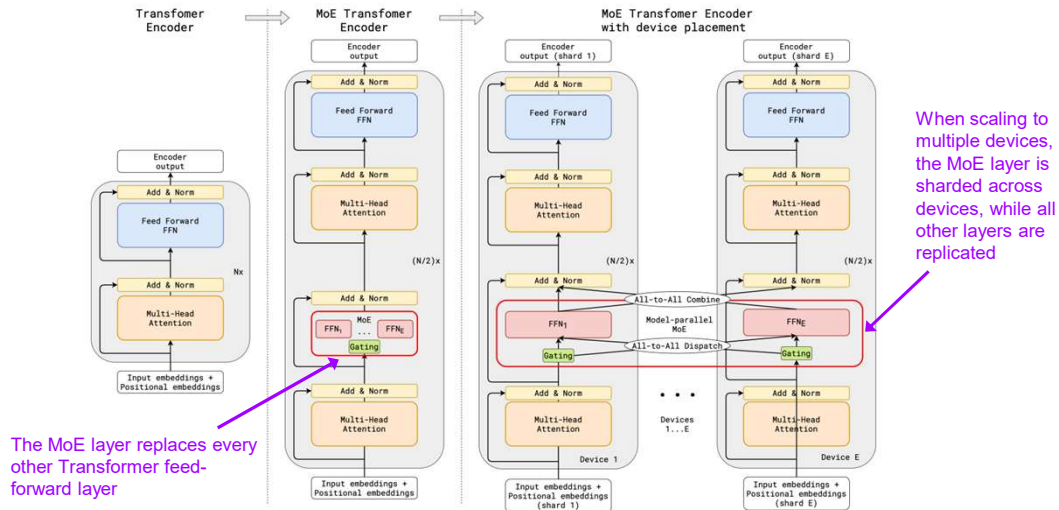
$$\text{MoE output} \rightarrow f(x) = \sum w_i \times h_i(x)$$

Weight assigned to the  $i$ -th expert by the gating network, indicating its relevance to the input

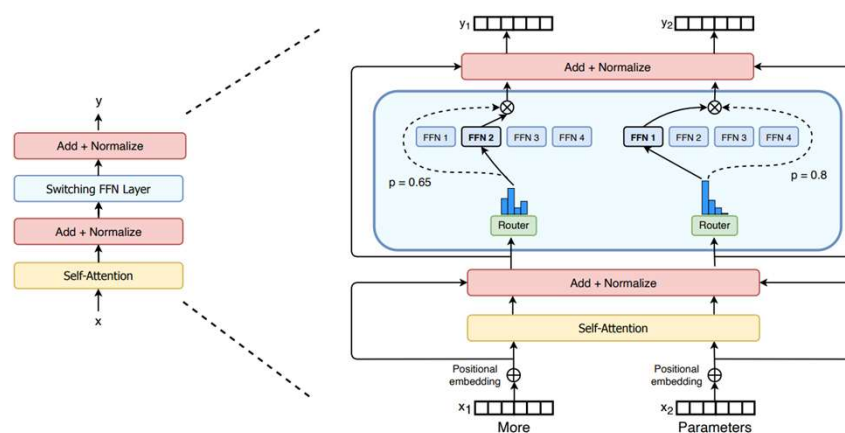
$$\text{SparsityRatio} = \frac{|\text{Experts}| - \sum |w_i|}{|\text{Experts}|}$$

How effectively the gating network selects a small subset of experts for each input, leading to computational savings and potentially improved generalization

## MoE: GShard [\[Lepikhin et al., 2020\]](#)



## MoE: Switch Transformer [\[Fedus et al., 2022\]](#)



## MoE: a learned gating mechanism

- The **gating network** consists of learned parameters that are pretrained simultaneously with the rest of the model
  - In its simplest form, the router is a simple linear
  - To improve **stability** and exploration, **noise** is often added to the routing scores before the top-k selection, ensuring that experts continue to be refined throughout training
  - Modern implementations utilize specific techniques like **Router Z-loss** to handle the high dynamic range of routing logits

$$y = \sum_{i=1}^n G(x)_i E_i(x)$$

Projection of the input hidden vector  $x$  through the  $i$ -th Expert's weights

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k))$$

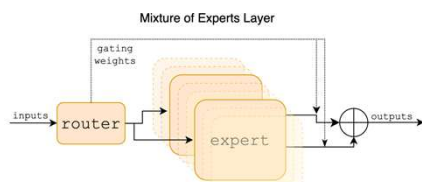
Noise perturbation and load balancing are added to prevent routing collapse to a single expert

## MoE: the unexpected logic of Expert Domains

- Expert specialization in MoE is **left entirely free** during training – the model is never "told" to create a math expert or a grammar expert
  - Instead of high-level topics, experts naturally gravitate toward **functional patterns**

| Expert specialization                                      | Expert position    | Routed tokens                                                                                                                                                              |
|------------------------------------------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sentinel tokens                                            | Layer 1            | been <extra.id.4><extra.id.7>floral to<br><extra.id.10><extra.id.12><extra.id.15><br><extra.id.17><extra.id.18><extra.id.19>...                                            |
|                                                            | Layer 4            | <extra.id.0><extra.id.1><extra.id.2><br><extra.id.4><extra.id.6><extra.id.7><br><extra.id.12><extra.id.13><extra.id.14>...                                                 |
|                                                            | Layer 6            | <extra.id.0><extra.id.4><extra.id.5><br><extra.id.6><extra.id.7><extra.id.14><br><extra.id.16><extra.id.17><extra.id.18>...                                                |
| Punctuation                                                | Layer 2<br>Layer 6 | ..... )<br>..... , & , & ? & , , ? , , , <extra.id.27>                                                                                                                     |
| Conjunctions and articles                                  | Layer 3<br>Layer 6 | The the the the the the the the the the<br>the the The the the the<br>a and and and and and or and a and .<br>the the if ? a designed does been is not                     |
| Verbs                                                      | Layer 1            | died falling identified fell closed left posted lost felt<br>left said read miss place struggling falling signed died<br>falling designed based disagree submitted develop |
| Visual descriptions<br><i>color, spatial position</i>      | Layer 0            | her over her know dark upper dark outer<br>center upper blue inner yellow raw mama<br>bright bright over open your dark blue                                               |
| Proper names                                               | Layer 1            | A Mart Gr Mart Kent Med Cor Tri Ca Mart<br>R Mart Lorraine Colin Ken Sam Ken Gr Angel A<br>Dou Now Ga GT Q Ga C Ko C Ko Ga G                                               |
| Counting and numbers<br><i>written and numerical forms</i> | Layer 1            | after 37 19. 6. 27 11 Seven 25 4. 54 I two dead we<br>Some 2012 who we few lower each                                                                                      |

## MoE: Mixtral [\[Jiang et al., 2024\]](#)



| Model        | Active Params | MMLU         | HellaS       | WinoG        | PIQA         | Arc-e        | Arc-c        | NQ           | TriQA        | HumanE       | MBPP         | Math         | GSM8K        |
|--------------|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| LLaMA 2 7B   | 7B            | 44.4%        | 77.1%        | 69.5%        | 77.9%        | 68.7%        | 43.2%        | 17.5%        | 56.6%        | 11.6%        | 26.1%        | 3.9%         | 16.0%        |
| LLaMA 2 13B  | 13B           | 55.6%        | 80.7%        | 72.9%        | 80.8%        | 75.2%        | 48.8%        | 16.7%        | 64.0%        | 18.9%        | 35.4%        | 6.0%         | 34.3%        |
| LLaMA 1 33B  | 33B           | 56.8%        | 83.7%        | 76.2%        | 82.2%        | 79.6%        | 54.4%        | 24.1%        | 68.5%        | 25.0%        | 40.9%        | 8.4%         | 44.1%        |
| LLaMA 2 70B  | 70B           | 69.9%        | <b>85.4%</b> | <b>80.4%</b> | 82.6%        | 79.9%        | 56.5%        | 25.4%        | <b>73.0%</b> | 29.3%        | 49.8%        | 13.8%        | 69.6%        |
| Mistral 7B   | 7B            | 62.5%        | 81.0%        | 74.2%        | 82.2%        | 80.5%        | 54.9%        | 23.2%        | 62.5%        | 26.2%        | 50.2%        | 12.7%        | 50.0%        |
| Mixtral 8x7B | 13B           | <b>70.6%</b> | 84.4%        | 77.2%        | <b>83.6%</b> | <b>83.1%</b> | <b>59.7%</b> | <b>30.6%</b> | 71.5%        | <b>40.2%</b> | <b>60.7%</b> | <b>28.4%</b> | <b>74.4%</b> |

- Mixtral 8x7B has the same decoder-only architecture of Mistral 7B, with the difference that each layer is composed of 8 feed-forward blocks (experts). For every input token vector, at each layer, a router network selects 2 experts to process the current state and combine their outputs (weighted sum).
- Even though each token only sees 2 experts, the selected experts can be different at each timestep. As a result, you may have **47B parameters** loaded in VRAM but, for efficiency, we only perform inference (floating point operations) on **13B** of them.
- Mixtral outperforms or matches LLaMA-2 70B performance on almost all popular benchmarks while using 5× fewer active parameters during inference.

## MoE: Phixtral [\[2x2\]](#) [\[4x2\]](#)



MoE on Microsoft's [Phi-2 models](#)

- It combines 2 to 4 fine-tuned models, each containing 2.8B parameters, surpassing the performance of individual experts
- Same MoE architecture of Mixtral

| Model                           | Average | AGIEval | GPT4All | TruthfulQA | Bigbench | Likes   |
|---------------------------------|---------|---------|---------|------------|----------|---------|
| mlabonne/phixtral-2x2_8         | 47.78   | 34.1    | 70.44   | 48.78      | 37.82    | 131 ❤️  |
| mlabonne/phixtral-4x2_8         | 47.7    | 33.91   | 70.44   | 48.78      | 37.68    | 191 ❤️  |
| abacaj/phi-2-super              | 47.16   | 31.95   | 70.81   | 48.39      | 37.49    | 72 ❤️   |
| lxuechen/phi-2-dpo              | 46.93   | 30.39   | 71.68   | 50.75      | 34.9     | 15 ❤️   |
| cognitivecomputations/dolphin-  | 46.89   | 33.12   | 69.85   | 47.39      | 37.2     | 177 ❤️  |
| meta-math/MetaMath-Mistral-7B   | 46.64   | 33.91   | 70.12   | 44.83      | 37.71    | 70 ❤️   |
| marcel/phixtral-4x2_8-gates-poc | 46.51   | 31.78   | 70.22   | 47.01      | 37.02    | 4 ❤️    |
| Yhyu13/phi-2-sft-dpo-gpt4_en-es | 46.43   | 30.61   | 71.13   | 48.74      | 35.23    | 8 ❤️    |
| deepseek-ai/deepseek-moe-16b-   | 45.72   | 30.42   | 68.72   | 48.73      | 35.02    | 96 ❤️   |
| g-ronimo/phi-2-OpenHermes-2.5   | 45.3    | 30.27   | 71.18   | 43.87      | 35.9     | 10 ❤️   |
| microsoft/phi-2                 | 44.61   | 27.98   | 70.8    | 44.43      | 35.21    | 2904 ❤️ |
| stabilityai/stablelm-zephyr-3b  | 44.42   | 34.04   | 62.07   | 46.46      | 35.11    | 216 ❤️  |
| alpindale/gemma-7b              | 33.6    | 20.67   | 38.48   | 46.66      | 28.58    | 5 ❤️    |
| google/gemma-7b                 | 33.56   | 20.64   | 38.49   | 46.61      | 28.51    | -1 ❤️   |

[\[Leaderboard\]](#)

## MoE: Gemini 1.5 Pro [\[Paper\]](#)



| Model summary             |                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Model architecture</b> | Gemini 1.5 Pro is a sparse mixture-of-expert (MoE) Transformer based model that builds on and scaling MoE vision/language models at Google ( <a href="#">Clark et al., 2020</a> ; <a href="#">Fedus et al., 2021</a> ; <a href="#">Lepikhin et al., 2020</a> ; <a href="#">Riquelme et al., 2021</a> ; <a href="#">Shazeer et al., 2017</a> ; <a href="#">Zoph et al., 2022</a> ). |
| <b>Input(s)</b>           | Text string (e.g., a question, a prompt, a document(s) to be summarized), video (up to one hour), images, audio files (up to 11 hours).                                                                                                                                                                                                                                            |
| <b>Output(s)</b>          | Generated text in response to the input (e.g., an answer to the question, a summary of multiple documents, comparing documents/videos).                                                                                                                                                                                                                                            |

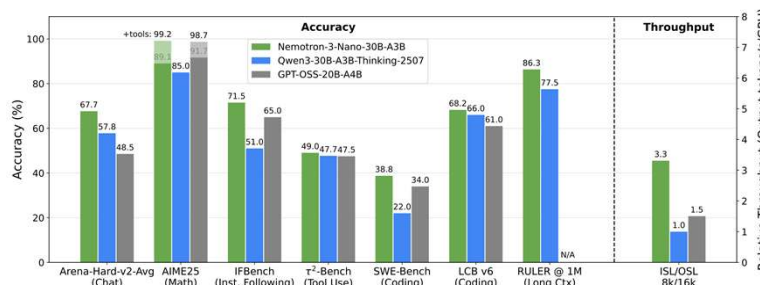
Gemini 1.5 Pro is a sparse mixture-of-expert (MoE) Transformer-based model that builds on Gemini 1.0's ([Gemini-Team et al., 2023](#)) research advances and multimodal capabilities. Gemini 1.5 Pro also builds on a much longer history of MoE research at Google ([Clark et al., 2022](#); [Du et al., 2022](#); [Fedus et al., 2021](#); [Lepikhin et al., 2020](#); [Riquelme et al., 2021](#); [Shazeer et al., 2017](#); [Zoph et al., 2022](#)) and language model research in the broader literature ([Anil et al., 2023](#); [Anthropic, 2023a](#); [Brown et al., 2020](#); [Chowdhery et al., 2023](#); [Hoffmann et al., 2022](#); [Jiang et al., 2024](#); [Kim et al., 2021](#); [OpenAI, 2023a,b](#); [Rae et al., 2021](#); [Raffel et al., 2020](#); [Roller et al., 2021](#); [Thoppilan et al., 2022](#); [Touvron et al., 2023a,b](#); [Vaswani et al., 2017](#)). MoE models use a learned routing function to direct inputs to a subset of the model's parameters for processing. This form of conditional computation ([Bengio et al., 2013](#); [Davis and Arel, 2014](#); [Jacobs et al., 1991](#)) allows models to grow their total parameter count while keeping the number of parameters that are activated for any given input constant.

## MoE: NVIDIA Nemotron 3 Nano [\[Paper\]](#)

Release date  
15 Dec. 2025

Nemotron 3 is the most efficient family of open models with leading accuracy for agentic AI.

- **Hybrid MoE:** Utilize a hybrid *Mamba-Transformer* MoE architecture to provide best-in-class throughput while having better or on-par accuracy than standard Transformers.
- **Long Context:** Support context length up to 1M tokens.
- **Multi-environment RL Post-training:** Trained using a diverse set of RL environments helping models achieve superior accuracy across a broad range of tasks.
- **Granular Reasoning Budget Control:** Trained to work with inference-time budget control.



## MoE: Pros and Cons

- Why MoE shines ✨
  - **Efficiency:** unlike traditional ensemble methods where all models contribute to every prediction, MoE only activates a few experts per input, enhancing computational efficiency and resource utilization
  - **Adaptability:** each expert continuously learns and improves within its niche, resulting in a constantly adapting model that can handle various input types effectively
  - **Scalability:** MoE can be easily scaled by adding more experts to address increasingly complex problems without significantly impacting computational requirements
- Main MoE challenges (other than model size increase 🦾 )
  - **Increased complexity:** designing and training the network components, particularly the gating network, can be more intricate compared to simpler models
  - **Interpretability:** understanding how MoE arrives at a specific prediction can be difficult due to the dynamic interaction between experts and the gating network

## MoE: Historical and Future Perspectives

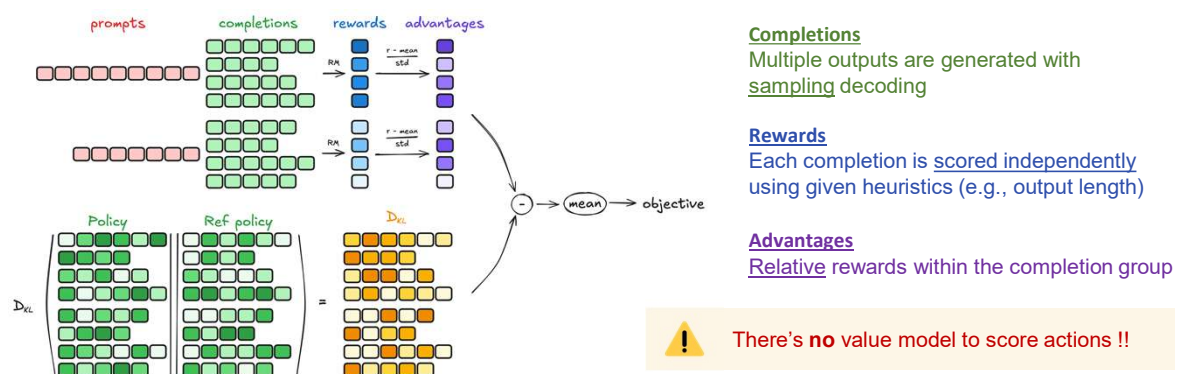
- 📖 A historical retrospective
  - While MoE is a recent buzzword, its roots trace back to the 1991 paper “Adaptive Mixture of Local Experts” [[Jacobs et al., 1991](#)]
  - This work proposed a framework where separate networks, each adept at handling distinct subspaces of the data, collaborated to achieve better results
  - Over the years, MoEs have seen renewed interest, fueled by advancements in deep learning and the growing need for scalable models with efficient training processes
- MoE without high-end GPUs
  - Thanks to offloading algorithms, it has become possible to run large MoE LLMs on consumer hardware with limited accelerator memory, e.g., Mixtral 8x7B on a free-tier Google Colab instance [[Eliseev and Mazur, 2023](#)]

## Outline

- Efficient Attention Mechanisms
- Parameter-Efficient Fine-Tuning
- Mixture of Experts
- **Optimization & Decoding Strategies** 
- Architectural Developments
- Multi-modal Language Models

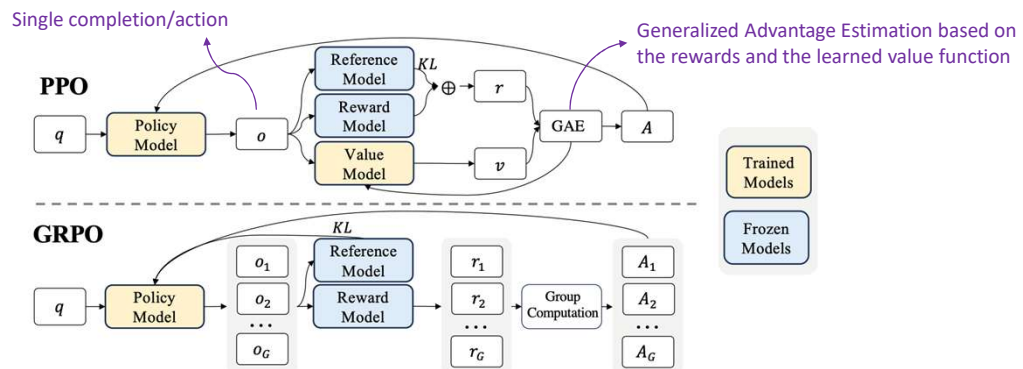
## Group Relative Policy Optimization (GRPO)

- **GRPO** ([Z. Shao et al., 2024](#)) operates as an online learning algorithm, refining parameters iteratively via trajectories generated by the model itself
- The objective seeks to maximize the **advantage** of generated completions subject to a constraint that limits divergence from the **reference policy**



## GRPO: no more space for value models

- By replacing the resource-heavy value model with **in-batch group scoring**, GRPO achieves a significant reduction in training resources



? Can we really consider GRPO an RL framework? Can **real exploration** be done **within-distribution**? Maybe we are simply trading pass@N with pass@1

## GRPO: training loss

- The objective is to **incentivize** completions that **maximize** the reward
  - Unlike supervised training, loss-backward is performed over the **entire** generated sequence

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[ \frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left( \frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] \right\}$$

Ensures that updates do not deviate excessively from the reference policy

$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$

Average over completions

Average over completion length

**DAPO** (Q. Yu et al., 2025) highlights a limitation of GRPO's sample-level loss in long-completion scenarios – longer texts are under-penalized due to noisier rewards. They propose **token-level** normalization of reward scores.

$$\mathcal{L}_{GRPO}(\theta) = -\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} l_{i,t}$$

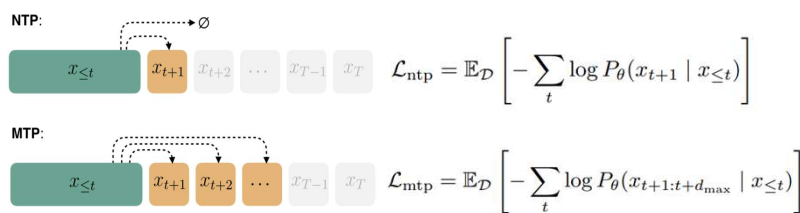
$$\mathcal{L}_{DAPO}(\theta) = -\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} l_{i,t}$$

## Multi-Token Prediction (MTP)

### ▪ Limits of Next-Token Prediction

- Next-token prediction (i.e., teacher forcing) can encourage "shortcuts" rather than learning
- Tasks requiring planning and distant dependencies (e.g., path-finding task on graphs) can end up with random-level accuracy

### ▪ **Predicting multiple tokens** enables a richer training signal, better information sharing between tokens, and **implicit planning in hidden states**



- MTP, as a pre-training objective, enhances some seq-to-seq tasks but poorly scales with depth
- Multi-Token with Registers (MuToR) (A. Gerontopoulos et al., 2025) improves MTP effectivity during SFT by inserting learnable register tokens into the input

## Speculative decoding: inefficient recursive generation

- Standard text generation with LLMs requires **recursive forward passes** to calculate the next token's probability
  - This causes **high latency** due to the **impossibility to batch** sequential operations

"The quick brown"



For more insights, check out: [Assisted Generation](#), [Blockwise Parallel Decoding](#), [Speculative Sampling](#)

## Speculative decoding: beyond the single token horizon

- **Speculative decoding** overcomes the sequential bottleneck by using a smaller "draft" model to predict multiple future tokens at once
  - This allows the main model to **verify** the draft in a **single parallel pass**, accepting the sequence only up to the first prediction mismatch

"The quick brown"

## Outline

- Efficient Attention Mechanisms
- Parameter-Efficient Fine-Tuning
- Mixture of Experts
- Optimization & Decoding Strategies
- **Architectural Developments** 
- Multi-modal Language Models

## The Big LLM Architecture Comparison

- It has been several years since the original GPT architecture was developed
- At a first glance, looking back at [GPT-2 \(2019\)](#) and forward to [LLaMA 4 \(2025\)](#), one might be surprised at how structurally similar these models still are
  - *Have we truly seen groundbreaking changes ?*
- In this lecture, we will show you that we have not hit a wall with LLMs, as [research groups have still not converged on the same architectures](#)
- **Note:** comparing LLMs to determine key ingredients that contribute to their good (or not-so-good) performance is notoriously challenging
  - Datasets, training techniques, and hyperparams vary widely and are often not documented



For this part, we will partially rely on the analysis by Sebastian Raschka [\[Source\]](#)

## Outline

- Efficient Attention Mechanisms
- Parameter-Efficient Fine-Tuning
- Mixture of Experts
- Optimization & Decoding Strategies
- **Architectural Developments**
- Multi-modal Language Models

- **Positional Embedding** 
- Attention in the latest LLMs

## Positional Encoding: A Recap

- Transformer models process all input tokens in **parallel**
  - That parallelism is what makes them so powerful — but it also means that, by default, a model has **no inherent sense of sequence: it must be added**
- Labeling each input token embedding with a unique position embedding is not enough: there must be a **linear translation property** across position embeddings
  - The absolute position of a token in the input sequence (e.g., 157<sup>th</sup>) *rarely* matters for **meaning** – **what matters is how tokens relate to each other (i.e., relative positions)**
  - Positional embeddings are especially needed for the attention blocks
  - “How far apart tokens are, not where they are”*

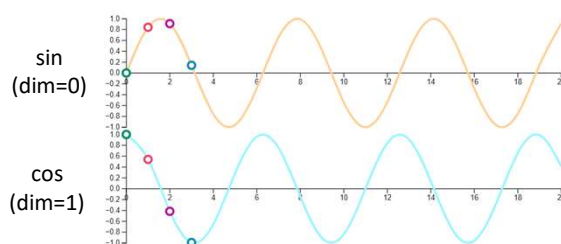
-2   -1   0   1   2  
The dog **chased** another dog

- To allow the model to learn relative positional patterns, **moving from the encoding of position  $p$  to  $p + k$  should be easy** (i.e., require just a linear transformation)

## Sinusoidal Positional Encoding: A Recap

- Early Transformer models solved this by **adding pre-defined sinusoidal positional encodings to the token embeddings**
  - Embedding dimensions are linked to sinusoids with decreasing frequency
  - Sine and cosine functions are alternated along embedding dimensions
  - Shifting the token position shifts the phase of all these sinusoids, i.e., where you are in the sine/cosine cycles; the overall position embedding is a multi-frequency signature
  - Relative patterns are encoded in the relative offsets in the phase differences

Example with  
embedding  
dimension = 2 (just  
one pair of sine and  
cosine with the  
same frequency)



| p0    | p1    | p2     | p3     |
|-------|-------|--------|--------|
| 0.000 | 0.841 | 0.909  | 0.141  |
| 1.000 | 0.540 | -0.416 | -0.990 |

## Sinusoidal Positional Encoding: Why Sine + Cosine

- You have probably always wondered: *Why alternate sine and cosine ? E.g., Wouldn't it have been enough to just use the sine function ?*
- One sinusoid (e.g., sine only) does not allow linear translation
  - The transformation matrix  $\mathbf{M}_k$  for  $\sin(p) \rightarrow \sin(p + k)$  requires us to know  $\cos(p)$ ; i.e., the values inside  $\mathbf{M}_k$  would have to change at every single token position

$$\sin(p + k) = \mathbf{M}_k \cdot \sin(p) \quad \mathbf{M}_k = \frac{\sin(p) \cos(k) + \cos(p) \sin(k)}{\sin(p)}$$

- With both sine and cosine, the shift becomes a simple linear rotation
  - We treat position as a vector  $(x, y)$  rather than a single number

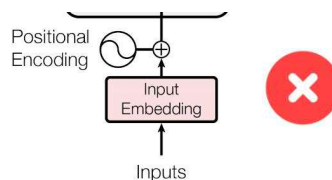
$$\begin{bmatrix} \sin(\omega_i(p + k)) \\ \cos(\omega_i(p + k)) \end{bmatrix} = \mathbf{M}_k \cdot \begin{bmatrix} \sin(\omega_i p) \\ \cos(\omega_i p) \end{bmatrix} \quad \mathbf{M}_k = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix}$$

✓ Constant w.r.t.  $p$ .  
It works identically whether you are at the start of the sequence or the end

- Consider a sequence of sine and cosine pairs, each associated with a frequency  $\omega_i$
- A rotation matrix can shift these sinusoidal functions by a fixed offset  $k$

## Sinusoidal Positional Encoding: Issues Emerged Over Time

- They **work well only within the sequence lengths seen during training**
  - At test time with longer sequences, we encounter new phase patterns the network has never seen, so attention/MLP layers are operating off-distribution
  - They provide the *capacity* for relative-position behaviors, but do not guarantee them—the model *may* learn them or absolute-position patterns (memorization)
- Element-wise addition mixes the **distance information** signal with **semantic information (interference)**



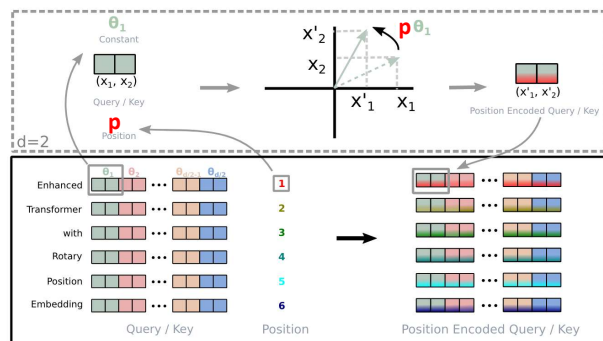
## RoPE: Rotatory Positional Embedding [Su et al., 2023]

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- RoPE integrates positional information directly into the **attention mechanism**
  - It **rotates the query  $q$  and key  $k$  vectors by a position-dependent angle**; early-tokens  $\rightarrow$  small rotation (higher dot-product = higher attention), later-tokens  $\rightarrow$  bigger rotation
  - In this way: (i) **the original token embeddings are preserved**; (ii) **the attention between two tokens depends on their relative angle** (how far apart they are)

### Multi-frequency rotation trick ① ② ③

- RoPE does not use a single rotation-rate for the whole embedding
- Instead, it uses **multi-frequency rotations** to keep both local (nearby token) and global (far token) relationships distinguishable
- Each pair (2D subspace) of embedding dimensions uses a different, constant base rotation frequency ( $\theta_i$ ) to prevent distant positions from collapsing to similar angles after exceeding the cycle length



## RoPE: Math Intuition

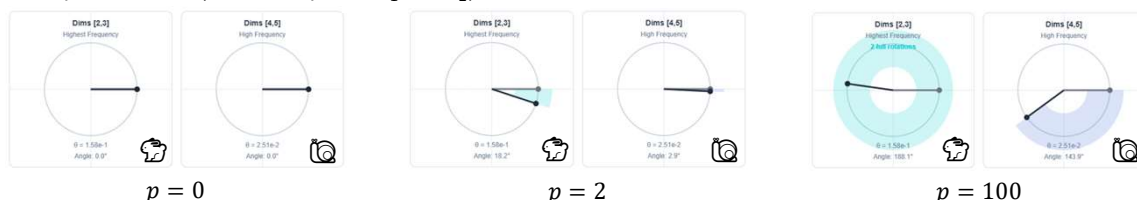
- For the full  $d$ -dimensional embedding, RoPE applies the following rotation

$$R = \begin{pmatrix} \cos(p\theta_1) & -\sin(p\theta_1) & & 0 & 0 \\ \sin(p\theta_1) & \cos(p\theta_1) & & 0 & 0 \\ & & \ddots & & \\ & & & \cos(p\theta_{d/2}) & -\sin(p\theta_{d/2}) \\ & & & \sin(p\theta_{d/2}) & \cos(p\theta_{d/2}) \end{pmatrix}$$

where each  $\theta_i = 10.000^{-2(i-1)/d}$  for  $i \in \{1, 2, \dots, d/2\}$

Decreasing frequency (i.e., increasing wavelength)

Example with  $d = 4$  (i.e., 2 subspaces,  $\theta_1$  and  $\theta_2$ )



The multi-frequency representation allows the model to naturally learn that attention should decay with distance. The further apart the tokens are, the more their dimensions are misaligned, and their inner product smaller.

## RoPE: Improvements Over Sinusoidal PE

| Feature                                   | Sinusoidal (additive) | RoPE (rotational)  |
|-------------------------------------------|-----------------------|--------------------|
| Encoded into attention scores directly    | ✗ <i>indirect</i>     | ✓                  |
| Relative position emerges naturally       | ✓ <i>in principle</i> | ✓ <i>by design</i> |
| No interference with semantic embeddings  | ✗                     | ✓                  |
| Position encoding defined beyond training | ✓                     | ✓                  |

RoPE makes attention depend on **relative positions**

$$a_{p_1, p_2} = q_{p_1}^T k'_{p_2} \propto \cos((p_2 - p_1)\theta_i)$$



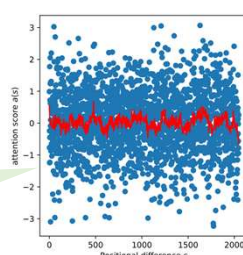
**RoPE has become the default choice in modern LLMs.**  
 (1) It accelerates training; (2) it better handles long-context.

## RoPE: Does Not Solve Length Extrapolation

- In theory, rotational differences hold even if context becomes longer...
- ... In practice, the model can still **overfit to trained relative distances**
  - During training, attention only sees relative spans up to the training window  $L$  (e.g., 2048)
  - So, it may learn weights that keep attention stable only in that range
  - **When tested with longer input texts**, the relative distance  $p_2 - p_1$  can become  $\gg L$
  - **Attention scores can explode or collapse**; the **perplexity may shoot up to very high numbers**, comparable to untrained models

Meta researchers analyzed RoPE behavior on LLaMA [Chen et al., 2023]

The attention score function learned during training appears well-behaved within  $[0, 2048]$



Beyond the training range, the function explodes to values over 8000 — causing catastrophic failure in attention computation

## RoPE: Position Interpolation [Chen et al., 2023]

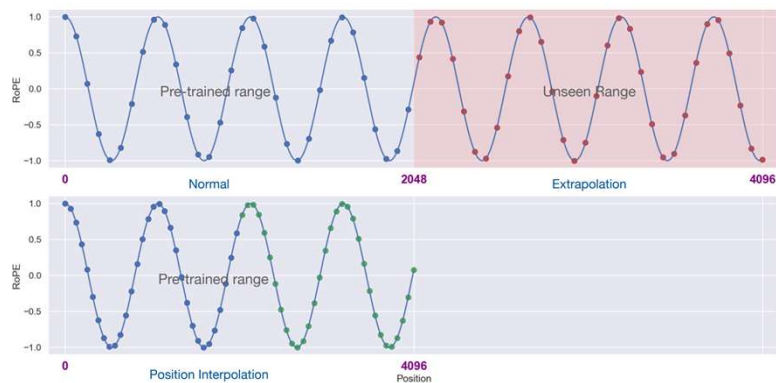
- Instead of asking the model to handle unseen positions (**extrapolation**), we can **compress** sequences to fit within the position range it knows (**interpolation**)
  - During inference with a max context length  $L_{context} > L_{train}$ , **every embedding dimension is equally scaled** by a factor  $L_{train}/L_{context}$

$$p' = p \cdot \frac{L_{train}}{L_{context}}$$

This ensures all positions map to  $[0, L_{train}]$ .

**Note:** it requires choosing a fixed scaling factor.

LLaMA models trained on 2K tokens could handle 8K-32K contexts with this simple technique



## RoPE: NTK-Aware Scaling [bloc97, 2023]

- Scaling every dimension indiscriminately by the same factor is suboptimal
  - High-freq RoPE dims should be scaled (compressed) **less** to keep **local sharpness**, i.e., the rotation denoting the smallest distance should not be too small for the model to detect it
  - Low-freq RoPE dims should be scaled (compressed) **more** to avoid them drifting into unseen ranges; the total rotation for **long-range** clocks should be similar to the training one
- We can **change the RoPE base frequency**

The original base, typically 10,000

$$\theta_i = \text{base}'^{-2(i-1)/d}$$

$$\text{base}' = \text{base} \times \alpha^{d/(d-2)}$$

$\alpha$  = a fixed scaling factor (e.g., 8 for 2K→16K context extension)

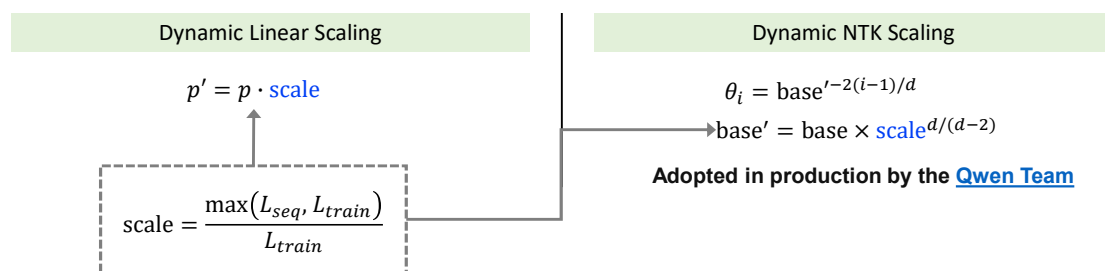
- By increasing the base, we slow down all rotation frequencies, and we slow low-frequency dimensions more than high-frequency ones
  - The elegance of NTK-aware scaling lies in recognizing that RoPE's rotation frequencies, not RoPE output positions themselves, are the right abstraction level for context extension

## RoPE: Dynamic Scaling [\[emozilla, 2023\]](#)

### ▪ The fixed scaling dilemma

- Both position interpolation and NTK scaling require choosing a fixed scaling factor upfront
- $\uparrow$  Choose a large factor: good for long sequences, but degrades short input performance
- $\downarrow$  Choose a small factor: preserves short sequence quality, but limits extension capability
- Forces an **unnecessary compromise before you even know what lengths you will process**

### ▪ Adjust the scaling factor dynamically based on the actual sequence length $L_{seq}$



## RoPE: YaRN [\[Peng et al., 2023\]](#)

- **Yet Another RoPE extension method** for supporting context windows much larger than those seen during pretraining, with minimal perplexity degradation
  - Other than modifying the positional encodings, it modifies the attention computation itself !
  - Consequently, it requires a lightweight fine-tuning stage (often <1% of pretraining compute)

### ▪ Two key innovations

1. Mix interpolation (scaling) and extrapolation (no scaling) depending on RoPE sinusoid frequencies (i.e., position embedding dimensions)
2. Attention temperature scaling

$$\text{softmax}\left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{\sqrt{d_k}}\right) \longrightarrow \text{softmax}\left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{t \sqrt{d_k}}\right)$$

- **Motivations.** Scaling compresses positional information, which can create artifacts where tokens get inflated scores.
- **Why adding temperature?** By softening the softmax (i.e., high  $t$ ), YaRN prevents the model from over-relying on a single, potentially incorrect high-scoring token. Instead, it forces the model to pay attention to more tokens.
- **TL;DR.** It deliberately makes attention «fuzzier» to handle compressed positions better.

## Outline

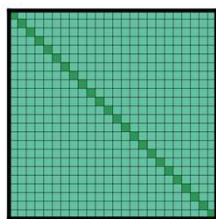
- Efficient Attention Mechanisms
- Parameter-Efficient Fine-Tuning
- Mixture of Experts
- Optimization & Decoding Strategies
- **Architectural Developments**
- Multi-modal Language Models

- Positional Embedding
- **Attention in the latest LLMs**

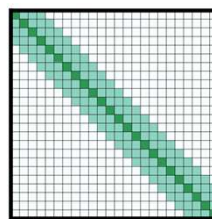


## Sliding Window Attention

- Each token attends only to **nearby tokens** within a **movable window**, avoiding full self-attention and reducing computational and memory cost.
- Efficiency trade-off: the sliding window captures local dependencies efficiently in long sequences, while limiting long-range interactions beyond the window.



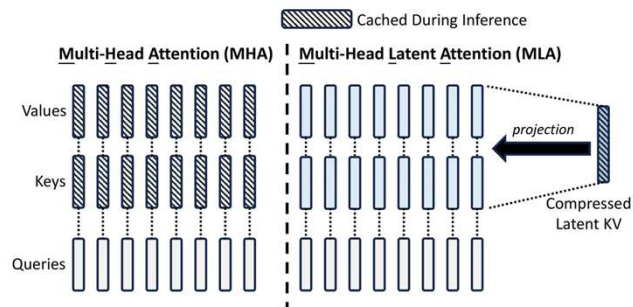
(a) Full  $n^2$  attention



(b) Sliding window attention

## Multi-Head Latent Attention (MLA)

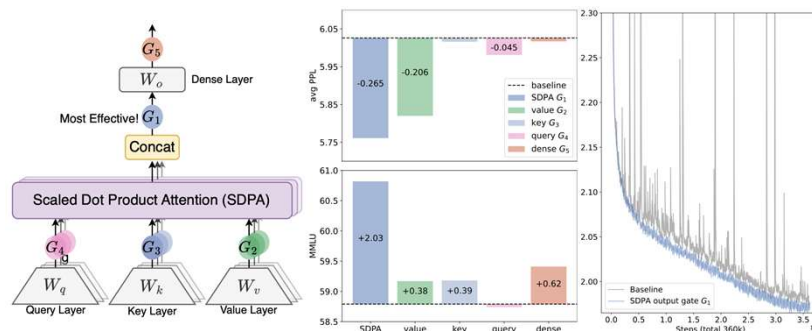
- MLA addresses the memory cost of Multi-Head Attention by **compressing** keys and values into a shared **latent representation**
- This reduces the KV cache size while preserving or improving performance



For more insights, check out: [DeepSeek V2, 2024](#); [DeepSeek V3, 2024](#); [DeepSeek R1, 2025](#)

## Gated Attention

- Gated Attention applies a learnable, input-dependent sigmoid gate immediately after the Scaled Dot-Product Attention output, introducing element-wise **sparsity** and **non-linearity** before the final output projection
- It yields profound stability improvements for large-scale training (eliminating loss spikes) and improves perplexity on 15B MoE and 1.7B dense models

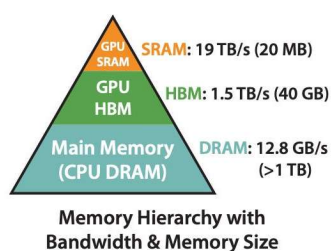


## Flash Attention [Dao et al., 2022]

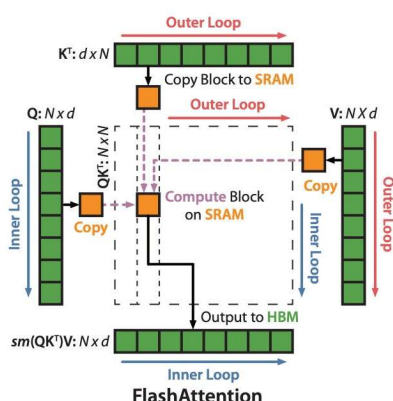
- Many efficient transformers reduce the complexity of self-attention to  $O(L)$ , but they fail to achieve **measurable speedups in practice**
- To solve this issue, FlashAttention **reformulates the self-attention operation in an efficient and IO-aware manner (drop-in replacement)**
  - The inner workings of FlashAttention are mostly **hardware-related**; it deeply optimizes CUDA memory access to achieve multi-fold speedups for inference and training
  - 🚀 Speeds up BERT-large training time by 15%
  - 🚀 Improves training speed by 3x for GPT-2
  - 🚀 Enables longer context lengths for LLMs
- After the [PyTorch 2.0 release](#), scaled dot product attention can be replaced with FlashAttention; for this reason many recent LLMs (e.g., Falcon) use it
  - [FlashAttention-2](#): modifies FlashAttention to yield further gains in efficiency
  - [FlashDecoding](#): an extension of FlashAttention that focuses upon improving *inference* efficiency in addition to training efficiency

## Flash Attention [Dao et al., 2022] : SRAM Execution

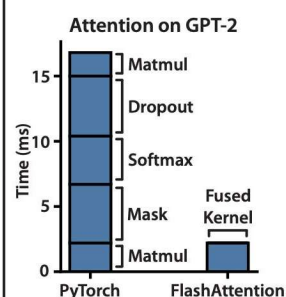
HBM = High Bandwidth Memory



Memory Hierarchy with Bandwidth & Memory Size

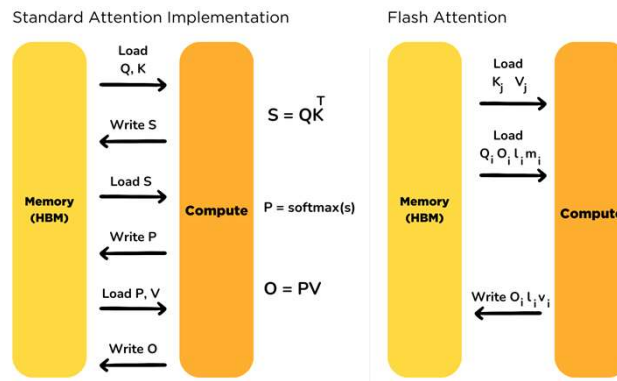


7.6x speedup



FlashAttention prevents materialization of the large  $L \times L$  attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FlashAttention loops through blocks of the  $K$  and  $V$  matrices and loads them to fast on-chip SRAM. In each block, FlashAttention loops over blocks of  $Q$  matrix (blue arrows), loading them to SRAM, and writing the output back to HBM.

## Flash Attention [Dao et al., 2022] : Grouping to Reduce Traffic



Flash-Attention deploys an efficient method for loading and storing data between the GPU computational unit and the VRAM, balancing the memory traffic. This involves [organizing attention heads into groups](#), enabling selective activation of specific sets of heads for each token.

## Multi and Grouped Query Attention

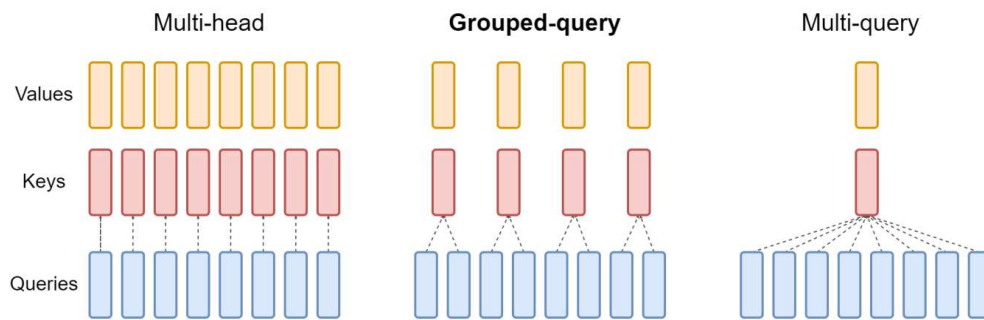
### ▪ Multi-query attention (MQA)

- Beyond FlashAttention, recent LLMs (e.g., Gemini, Falcon, PaLM) use MQA, an efficient multi-head self-attention implementation that [shares key and value projections between all attention heads in a layer](#) (instead of performing a separate projection for each head)
- This change does not make training faster but it significantly [improves inference speed](#)

### ▪ Grouped-query attention (GQA)

- Unfortunately, MQA can cause slight deteriorations in performance, which led some LLMs (e.g., LLaMA-2) to search for alternatives
- Instead of sharing all key/value projections across attention heads, GQA [divides the H total self-attention heads into groups](#) and [shares key/value projections within the same group](#)
- Such an approach is an interpolation between vanilla multi-head self-attention and MQA
- Interestingly, GQA maintains the performance of vanilla multi-head causal self-attention and achieves comparable efficiency to MQA

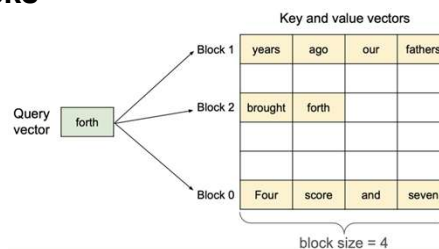
## Multi and Grouped Query Attention: Visual



[Ainslie et al., 2023]

## vLLM: Efficient Memory Management with Paged-Attention

- **Paged-Attention** algorithm allows for storing attention keys and values vectors in non-contiguous blocks in the memory
- Each KV block is a fixed-size contiguous chunk of memory that can store KV cache from left to right
- Attention for each new token is computed across all tokens in the **non-contiguous KV blocks**



✓ vLLM natively supports paged-attention and flash-attention, but it also implements Xformer techniques

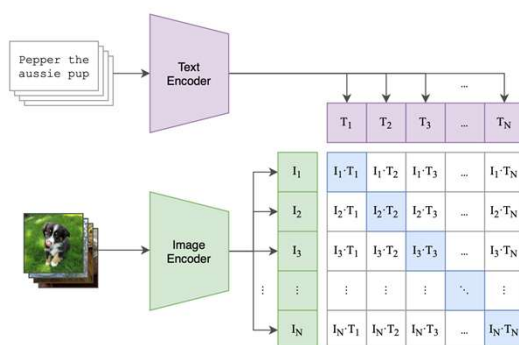
## Outline

- Efficient Attention Mechanisms
- Parameter-Efficient Fine-Tuning
- Mixture of Experts
- Optimization & Decoding Strategies
- Architectural Developments
- **Multi-modal Language Models** 

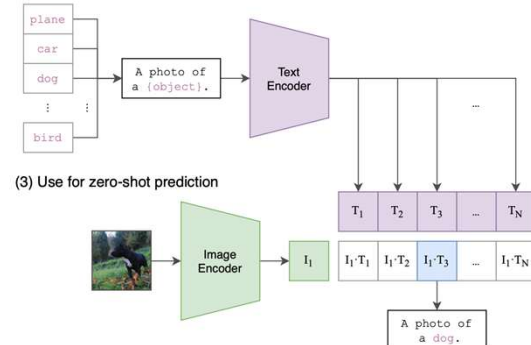
## Contrastive Pretraining Enables Zero-shot Classification

- CLIP-based models ([A. Radford et al., 2021](#)) align vision and language by jointly training image and text encoders on image-caption datasets
  - Contrastive learning maps images and texts into a shared feature space, minimizing the distance for matching pairs and maximizing it for non-matching ones

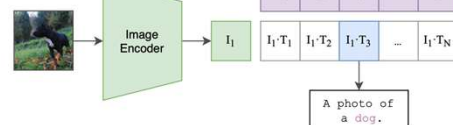
(1) Contrastive pre-training



(2) Create dataset classifier from label text

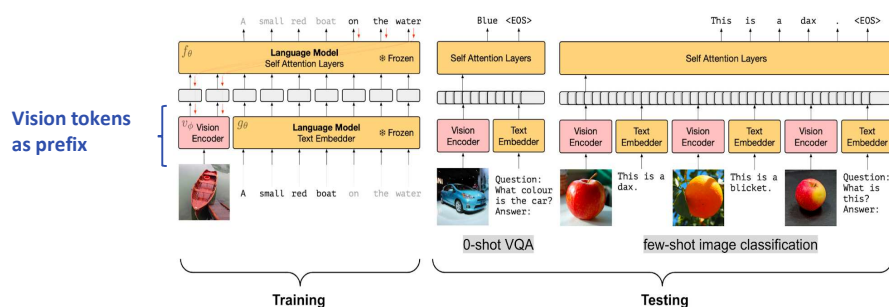


(3) Use for zero-shot prediction



## PrefixLM: image-conditioned text-generation

- Visual Transformers split images into patches treated as visual tokens
  - SimVLM** feeds a concatenation of image patches and **prefix** text to the encoder, while the decoder predicts the continuation of the text
  - Frozen** and **ClipCap** align image embeddings with a frozen language model and are trained on image-caption pairs to predict the next token
  - Models like **MAPL** and **Flamingo** keep both vision encoder and language model frozen, using resampling and cross-attention to fuse modalities



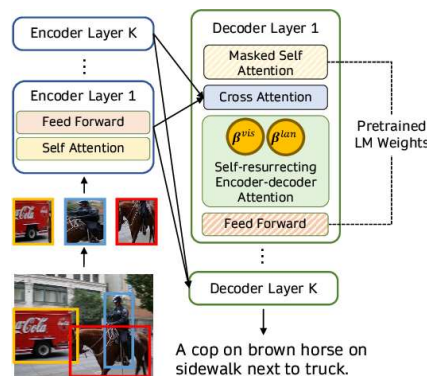
[disai-unibo-nlp.github.io](https://github.com/disai-unibo-nlp)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

81

## Fusing modality through Cross-Attention

- Models such as **VisualGPT** use a visual encoder to produce image embeddings that are injected via cross-attention into a PT language decoder
- More recent models like **FIBER** introduce **gated cross-attention** into both vision and language backbones, enabling more efficient multimodal fusion



[disai-unibo-nlp.github.io](https://github.com/disai-unibo-nlp)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

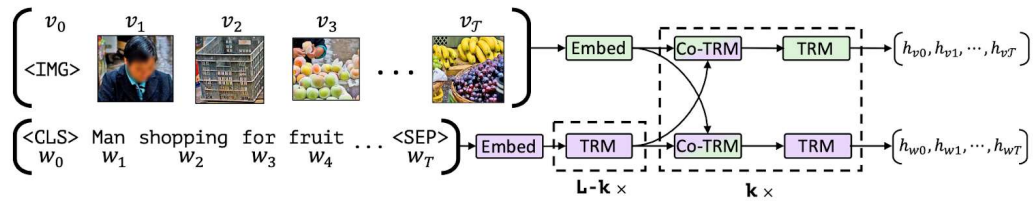
82

## Masked Language Modeling and Image Text Matching

Combining Masked Language Modeling (MLM) and Image–Text Matching (ITM) aligns fine-grained visual and textual representations, enabling tasks such as VQA, visual reasoning, image retrieval, and text-guided detection. This setup is used by models like VisualBERT, FLAVA, ViLBERT, LXMERT, and BridgeTower.

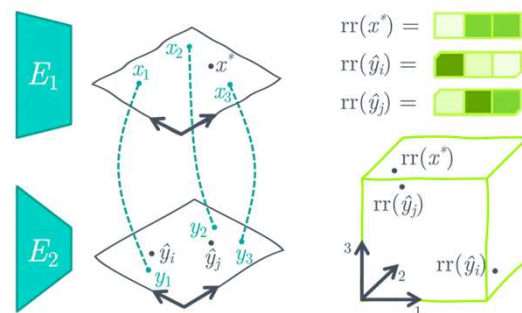
**MLM:** predict masked words in a caption using the associated image.

**ITM:** classify whether an image–caption pair matches, using randomly sampled negatives.



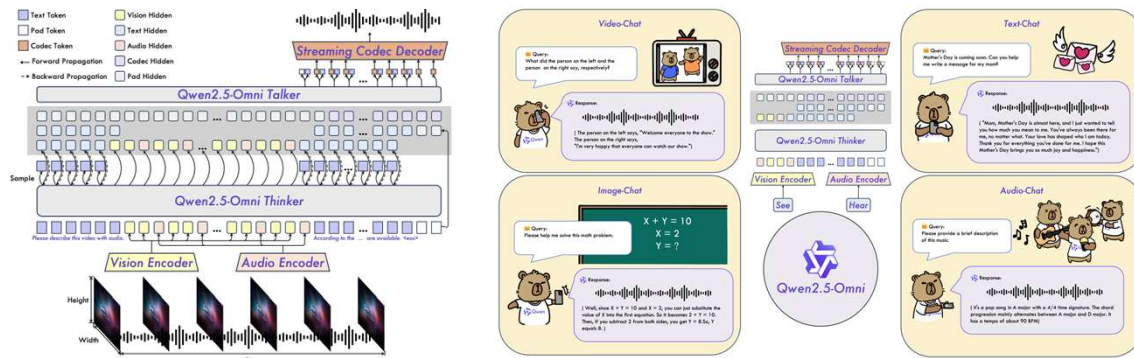
## Multimodal Language Models with no training

- ASIF proposes a simple way to convert pre-trained unimodal image and text models into a multimodal captioning system using a small image–text dataset and no additional training.
- It relies on the intuition that similar images have similar captions, enabling similarity-based retrieval in a learned relative representation space built from a few ground-truth multimodal pairs.



## Omni Models: beyond visual-text modalities

- Models like Qwen2.5-Omni are unified end-to-end multimodal model that processes text, audio, images, and video, and generates real-time text or speech responses



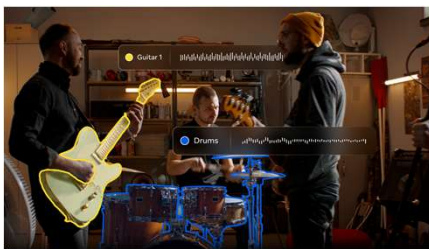
[disai-unibo-nlp.github.io](https://github.com/disai-unibo-nlp)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

85

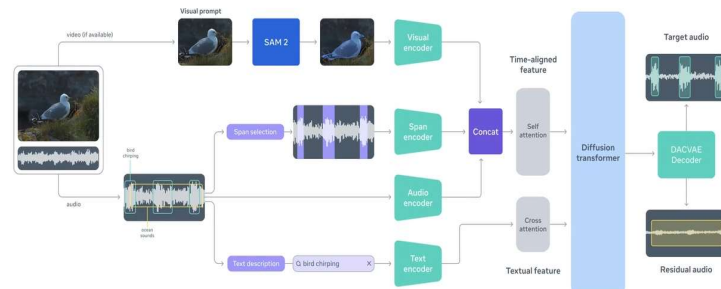
## SAM: Multimodal Model for Audio Separation

Release date  
16 Dec. 2025



- Generative modeling framework built on a **flow-matching diffusion transformer**
- In tandem, a second model is used, the Perception Encoder Audiovisual.
  - Extends Meta's Perception Encoder to audio-visual alignment.
  - Aligns frame-level video features with audio over time

- First model for segmenting sound through multimodal prompts [\[SAM Audio\]](#).
- Multimodal Prompting are:
  - Text prompts (e.g., "dog barking", "singing voice").
  - Visual prompts (e.g., by clicking on sounding objects).
  - Span prompts (e.g., by marking time segments where target audio occurs).



[disai-unibo-nlp.github.io](https://github.com/disai-unibo-nlp)

LLMs The latest upgrades - Moro G., Frisoni G., Molfetta L.

86



**Thanks for the attention.**  
**Is all you need.**